CS-302 Artificial Intelligence

WEEK 03

DR. SAMIA IJAZ

Organization of Lecture

1. Uninformed Search Strategies
    1. Breadth First Search (BFS)
    2. Depth First Search (DFS)
    3. Depth Limited Search Algorithm
    4. Uniform Cost Search Algorithm
    5. Bi-directional Search Algorithm
    6. Iterative Deepening Depth First Search Algorithm

Lecture Helping Material

https://www.youtube.com/watch?v=HXGaabF6T7U&list=PLV8vIYTIdSnYsdt0Dh9KkD9WFEi7nVgbe&index=12

1. AI Technique

1) **AI Technique**: It is a method that exploits knowledge that should be represented in such a way that:

i) Knowledge captures generalization
ii) Understandable by people
iii) Can easy be modified to correct errors.
iv) Can be used in many situations.
v) Can reduce its volume on its own (for example knowledge no more required should be minimized or reduced by AI technique itself).

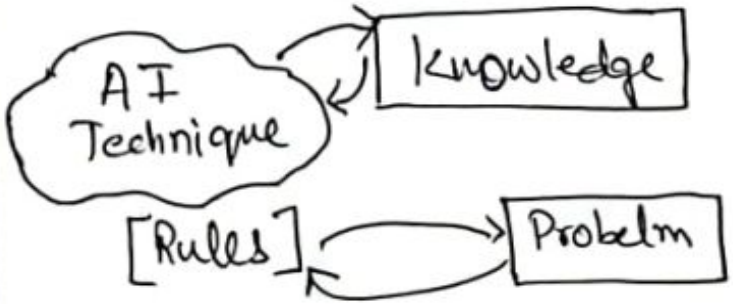
2. Parts of AI Technique

Parts of AI Technique:-

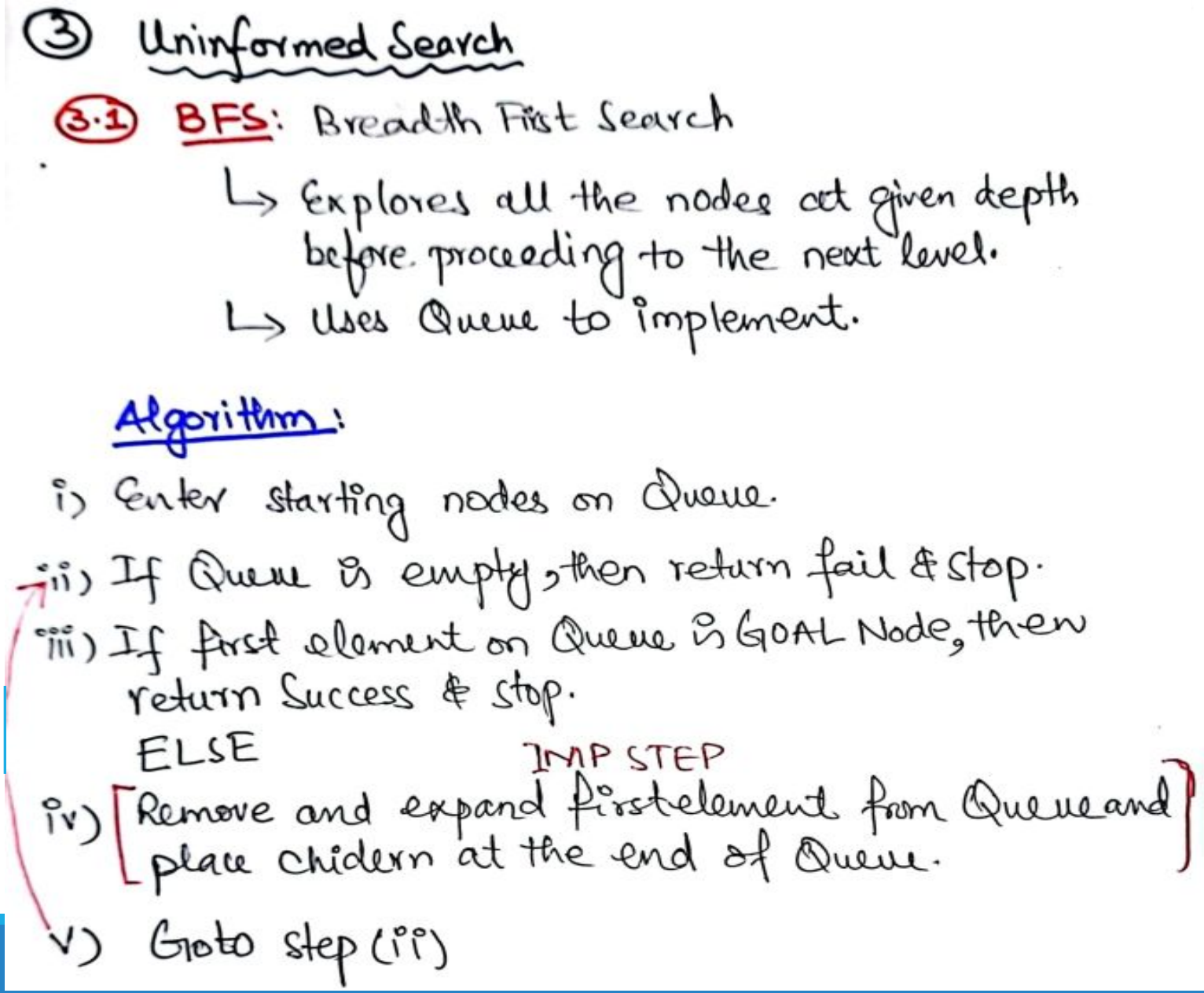
① **Knowledge Representation:** It is used to capture the knowledge about the real world. (Knowledge gathering from real world).

② **Search Algorithm:** Finding/Searching solution of the probelm.

Knowledge
→ Large in volume.
→ Not well formatted.
→ Constantly Changing.

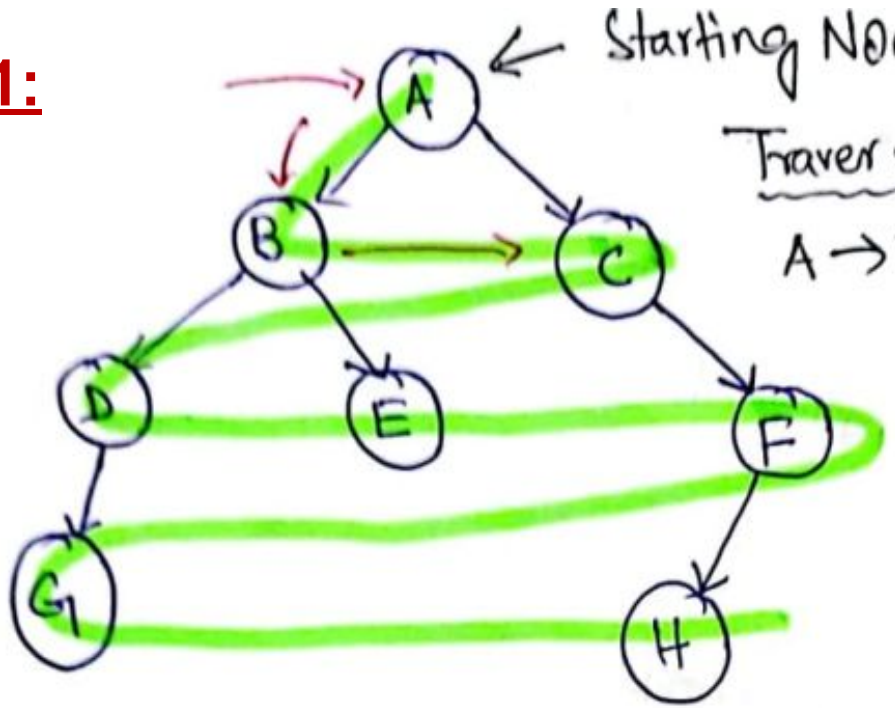


Uninformed Search Strategies (1/9)



Uninformed Search Strategies (2/9)

Example 1:



Starting Node.

Traversal

A → B → C → D → E → F → G → H

Initial Queue {A} = Goal node X.

↳ Remove from Queue and add its successor to Queue.

Uninformed Search Strategies (3/9)

→ Successors of A are B & C.

{B, C}

↳ Not Goal state/node, Remove Node B & expand it by placing its childernnodes (D&E) at the end of Queue.

{C, D, E}

↳ Remove C as it is not the Goalnode & same step as above.

{D, E, F}

↳ Removed D & same step as above.

{E, F, G}

Uninformed Search Strategies (4/9)

{E, F, G}

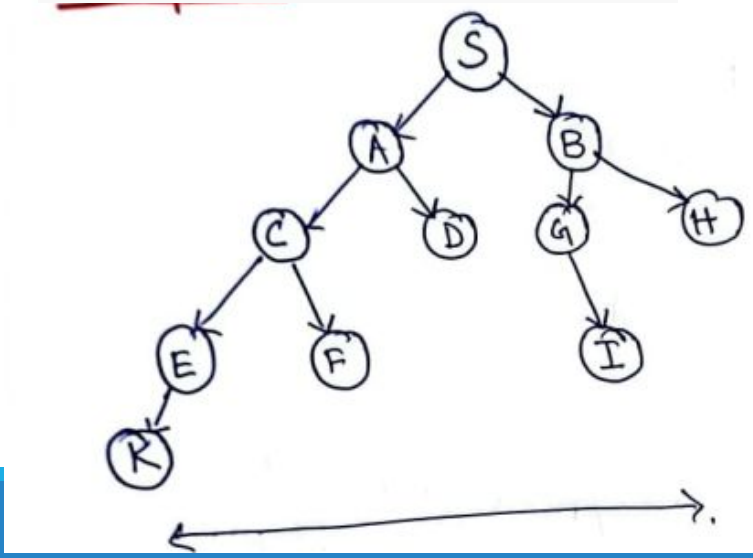
→ Remove E

{F, G}

→ Remove F

{G, H}

Class Activity



Uninformed Search Strategies (5/9)

Advantages: i) BFS will find a solution if it exists.
ii) It will try to find the minimal solution in least no. of steps.

Disadvantages:
i) It requires more memory (b/c tree is expand at every stage)
ii) Needs lots of time if solution is far from root node.

Time Complexity (T.C) & Space Complexity (S.C)

↳ $O(b^d)$ → depth
order of → branching factor

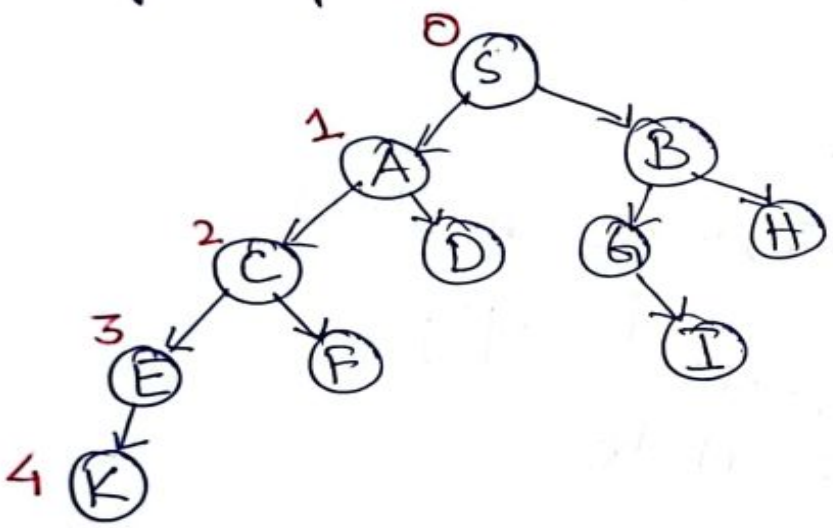
Uninformed Search Strategies (6/9)

Where b = branching factor (How many times a node is dividing?) in example 2

→ for example Node (S) is divided into two nodes A & node B.
therefore b = 2

d = depth.

→ for example depth of tree in example 2 is 4 i-e



Worst case time complexity for this particular example is: ?

Uninformed Search Strategies (7/9)

(3.2) **DFS**: Depth-First Search.

- Recursive Algorithm
- Starts from root node and follows each path to its greatest depth node before moving to the next path
- Implemented using STACK. [LIFO]

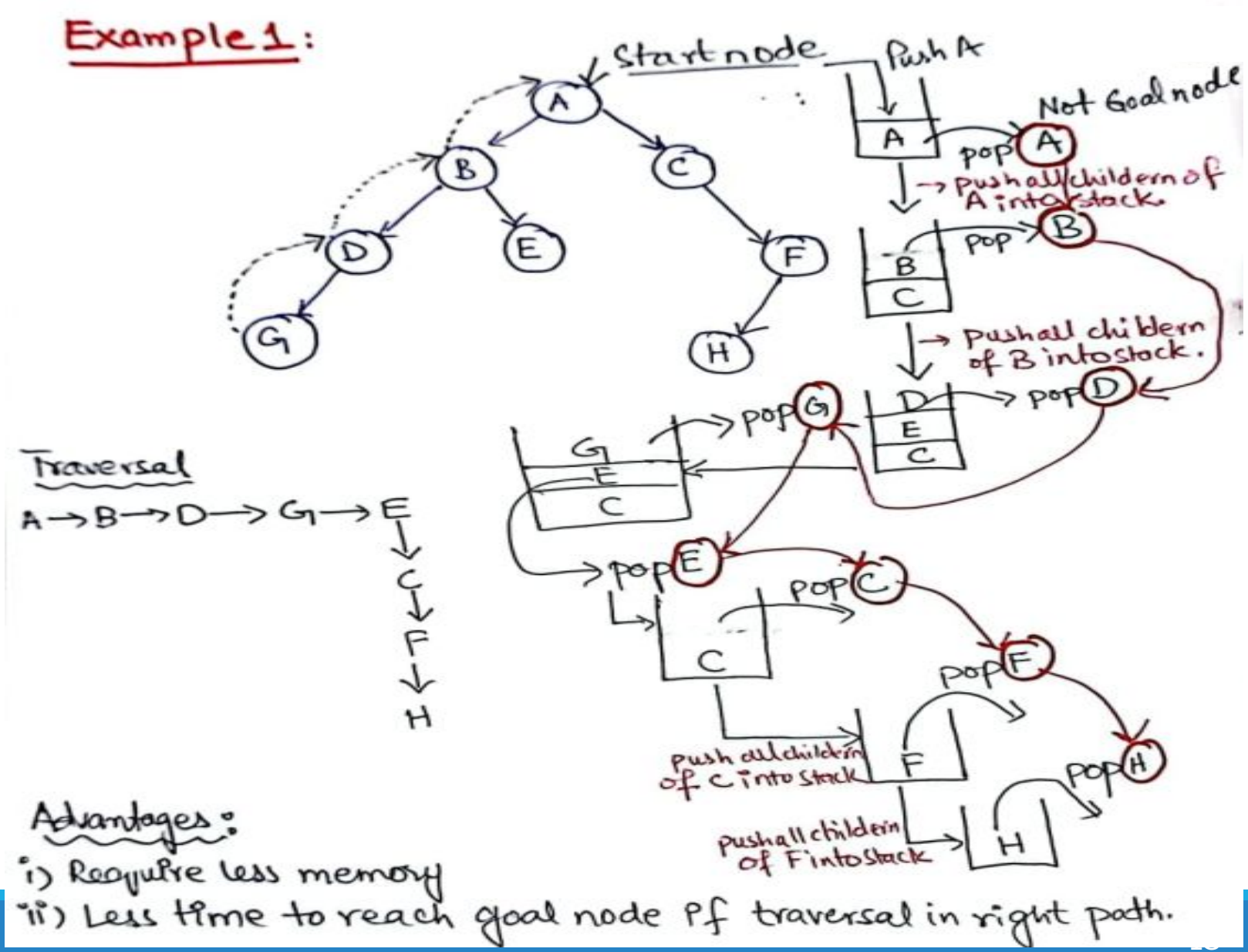
Algorithm → (PUSH)

i) Enter ROOT (START) node on STACK.

ii) Do until Stack is not empty

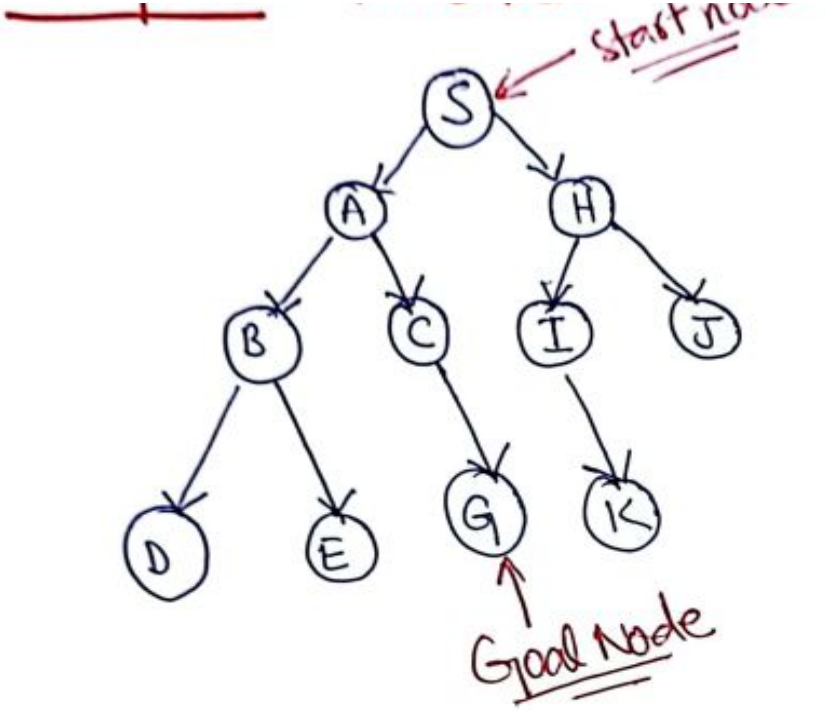
- a) Remove (POP) Node
  - i) If Node = Goal, stop
  - ii) Push all childern of node in Stack.

Uninformed Search Strategies (8/9)



Uninformed Search Strategies (9/9)

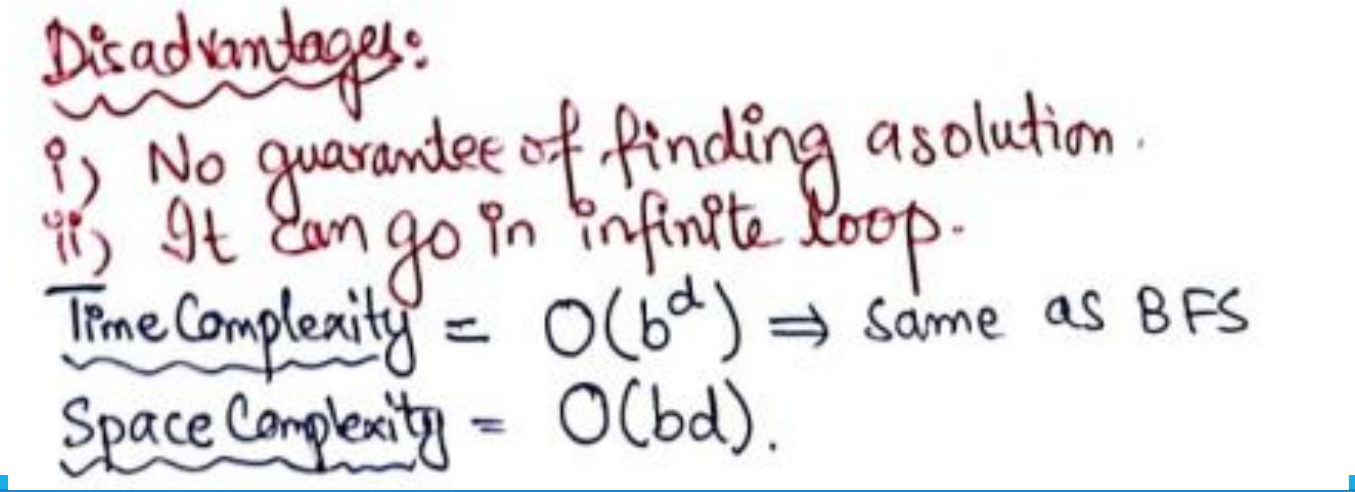
Class Activity



Uninformed Search Strategies (9/9)

Memory Efficient: DFS uses less memory as it stores only the nodes on the path from the root to the current node.

Simpler Implementation: Can be easier to implement recursively, especially when the depth of the tree is unknown.



Depth-Limited Search Algorithm (1/3)

3) Depth-Limited Search Algorithm:

↳ Working is similar to DFS but with a predetermined limit. (to avoid from Infinite loop).

↳ Helps in solving the probelm of DFS.

- DFS: Infinite Path

Termination Conditions:

- Cutoff: If **depth limit L is reached and more nodes remain unexplored**, return **cutoff**.

  Failure: If **all paths are fully explored** and the goal is **not found at any level**, return **failure**.

Depth-Limited Search Algorithm (2/3)

Advantages:

↳ Since we know the depth so less memory is required i.e, it is memory efficient.

Disadvantages:

↳ Can be terminated without finding solution. i.e., in case of 'Cutoff failure'. → Incompleteness

↳ Not optimal. It is not sure whether it will give you a best result.

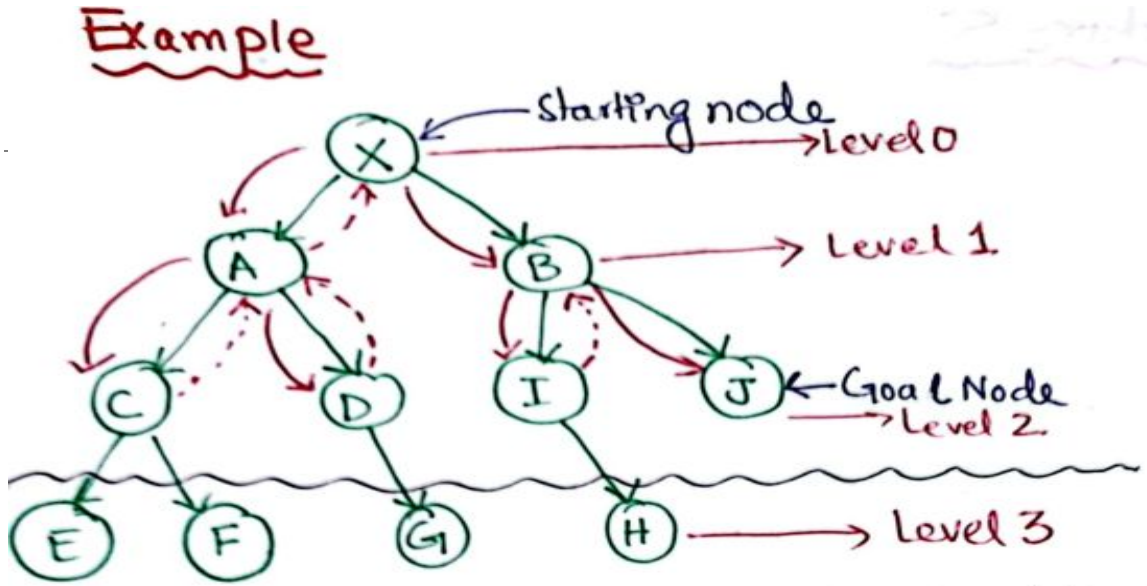
Time Complexity:

$T.C = O(b^d)$; same as DFS

Space Complexity:

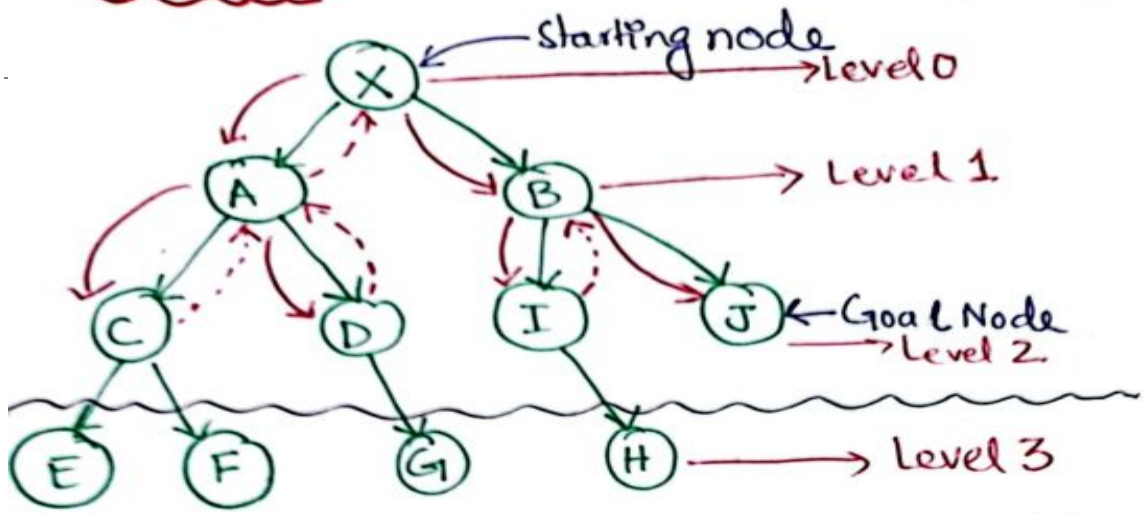
$S.C = O(bd)$; Same as DFS

Depth-Limited Search Algorithm (3/3)



Depth-Limited Search Algorithm (3/3)

Example



→ Let's say we take a predetermined depth as two, i.e.,
- [Depth, d = 2]

→ Traversal path

X → A → C → D → B → I — J

→ Let's say if we were given Goal node as 'H' with predetermined depth as 2. This depth will result in no solution. Because the solution is available at depth, d = 3. ⇒ Cutoff Failure Termination Condition.

Uniform Cost Search Algorithm (1/5)

4) Uniform Cost Search Algorithm

↳ Used for weighted Tree/Graph Traversal.

↳ Goal is to path finding to goal-node with lowest cummulative cost i.e., finding optimal path.

↳ Node expansion is based on path costs.

↳ Priority Queue is used for implementation.
- High priority to minimum cost path.

↳ Back Tracking can also be implemented in this algorithm.

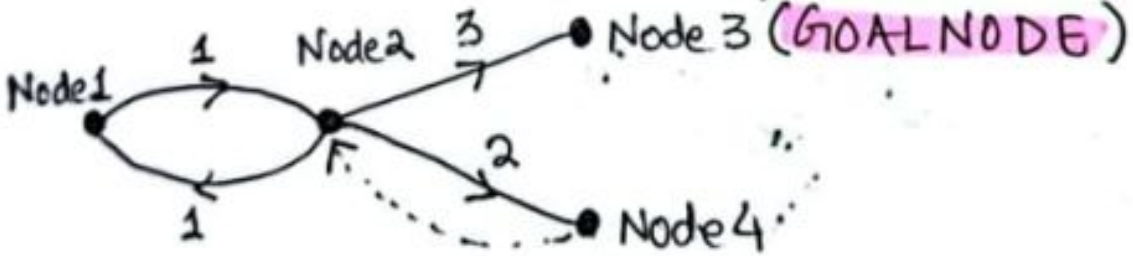
Uniform Cost Search Algorithm (2/5)

Advantages:

↳ It provides optimal solution. (Not Always).

Disadvantages:

↳ It can struck in Infinite loop.

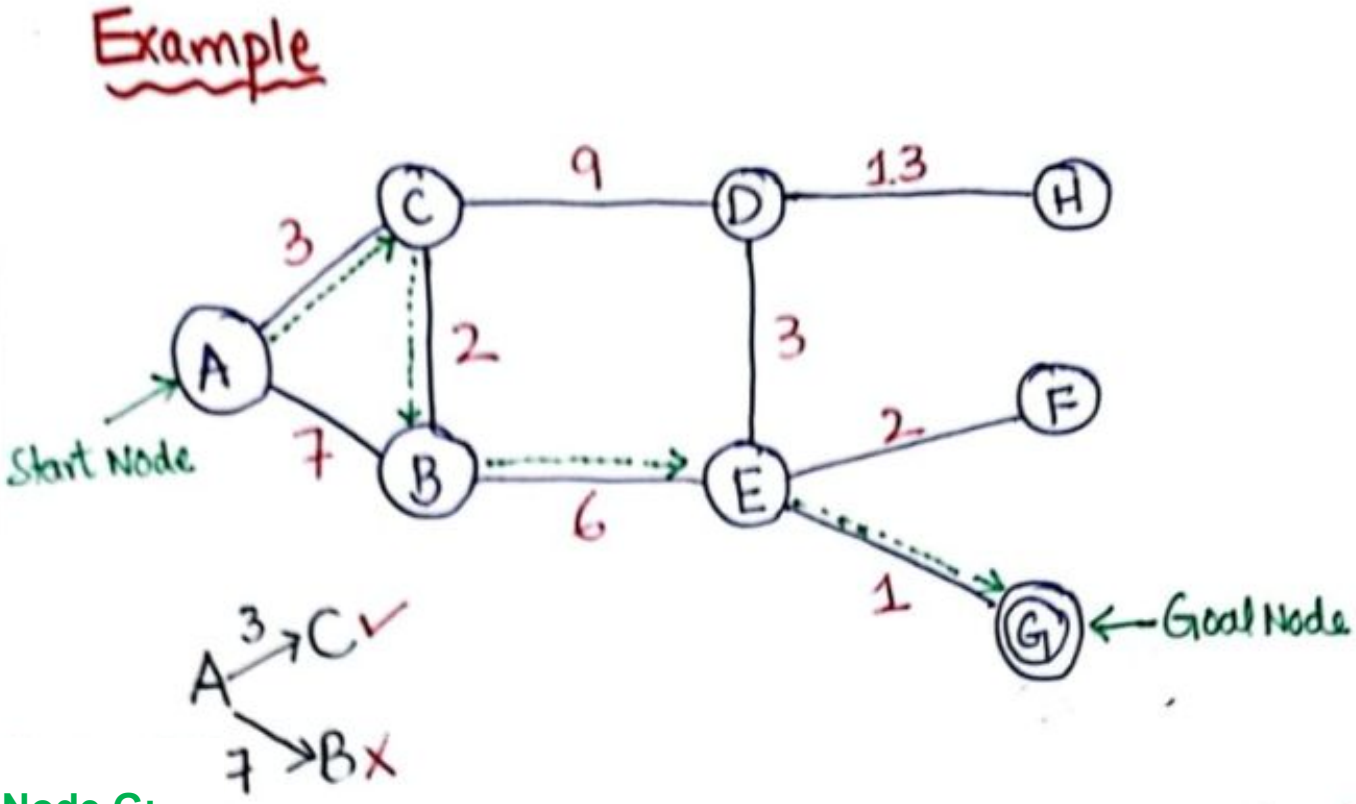


backtracking

Node 1 → Node 2 → Node 4 - - - - - → Node 2

Infinite loop

Uniform Cost Search Algorithm (3/5)



Goal Node G:

Path to Goal Node (G) ⟹ Backtracking is not involved.

A $\xrightarrow{3}$ C $\xrightarrow{2}$ B $\xrightarrow{6}$ E $\xrightarrow{1}$ G

Total Cost = 3+2+6+1 = 12

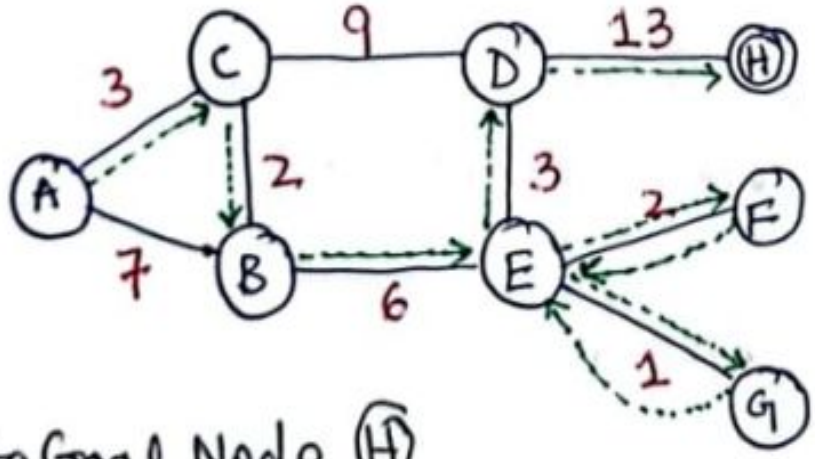
Uniform Cost Search Algorithm (4/5)

Goal Node H:

↳ Suppose Now select the Goal node as 'H'

↳ Backtracking will be involved.

↳ Redrawing above example:



Path to Goal Node (H)

$A \xrightarrow{3} C \xrightarrow{2} B \xrightarrow{6} E \xrightarrow{1} G \xrightarrow{1} E \xrightarrow{2} F \xrightarrow{2} E \xrightarrow{3} D \xrightarrow{13} H$

Total Cost = 3+2+6+1+1+2+2+3+13 = 33

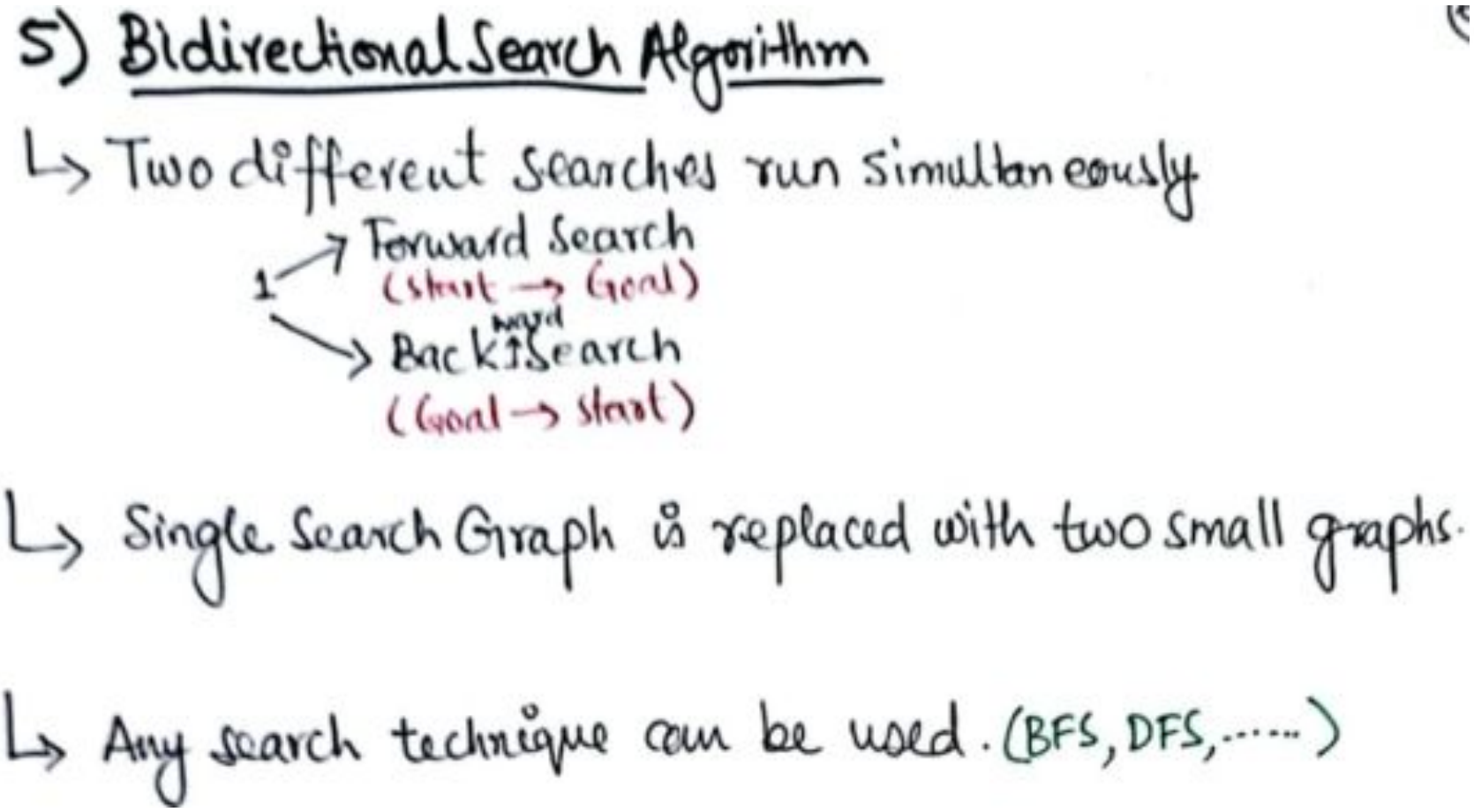
Uniform Cost Search Algorithm (5/5)

→ Sometimes we don't get optimal solution, this algorithm tries to find the optimal solution. For example for Goal Node 'H', the optimum path could be:

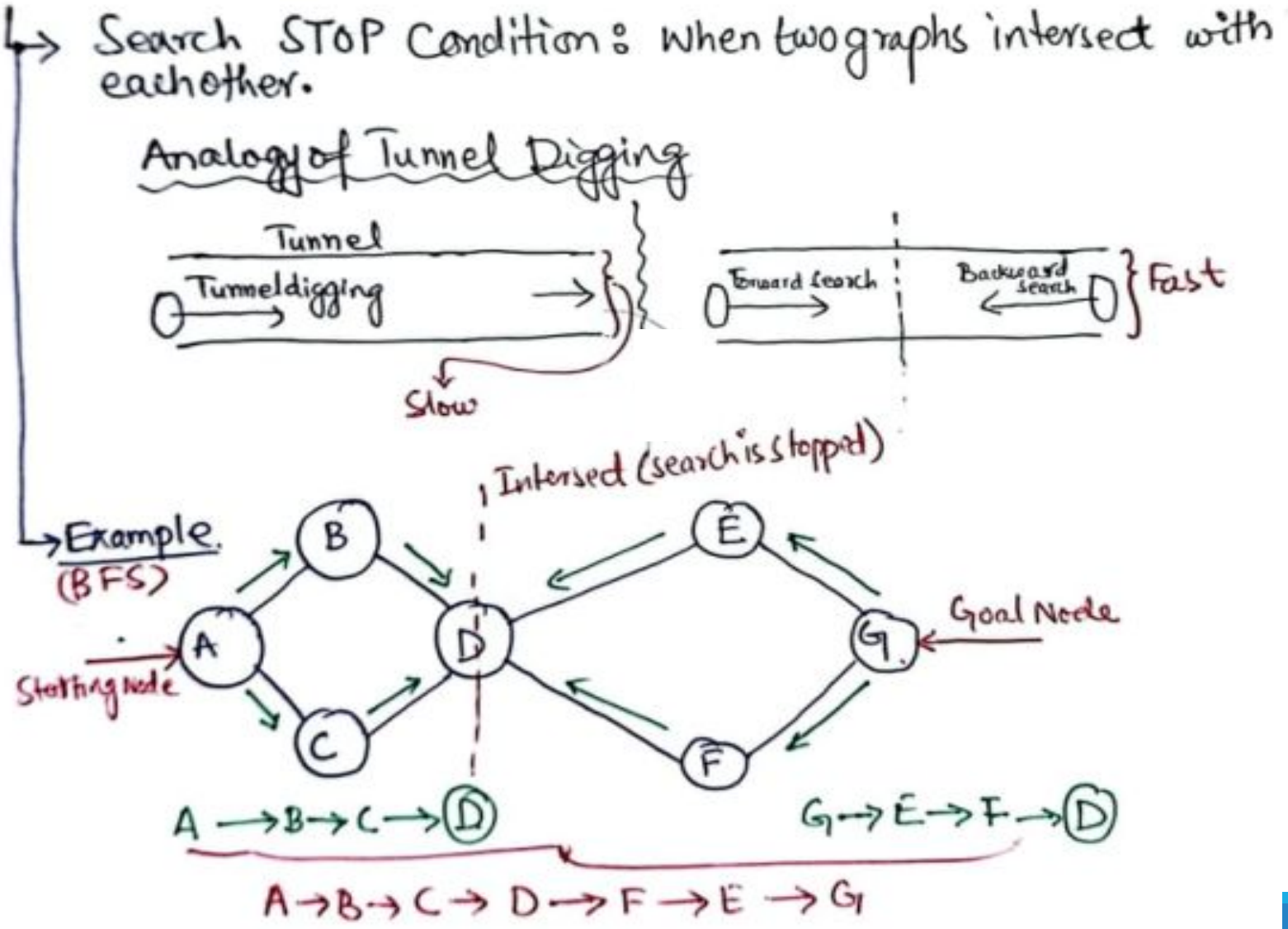
$$A \xrightarrow{3} C \xrightarrow{9} D \xrightarrow{13} H$$

Total Cost = $3+9+13 = 25 < 33$

Bi-directional Search Algorithm (1/4)



Bi-directional Search Algorithm (2/4)



Bi-directional Search Algorithm (3/4)

Advantages:

i) Fast searching technique.

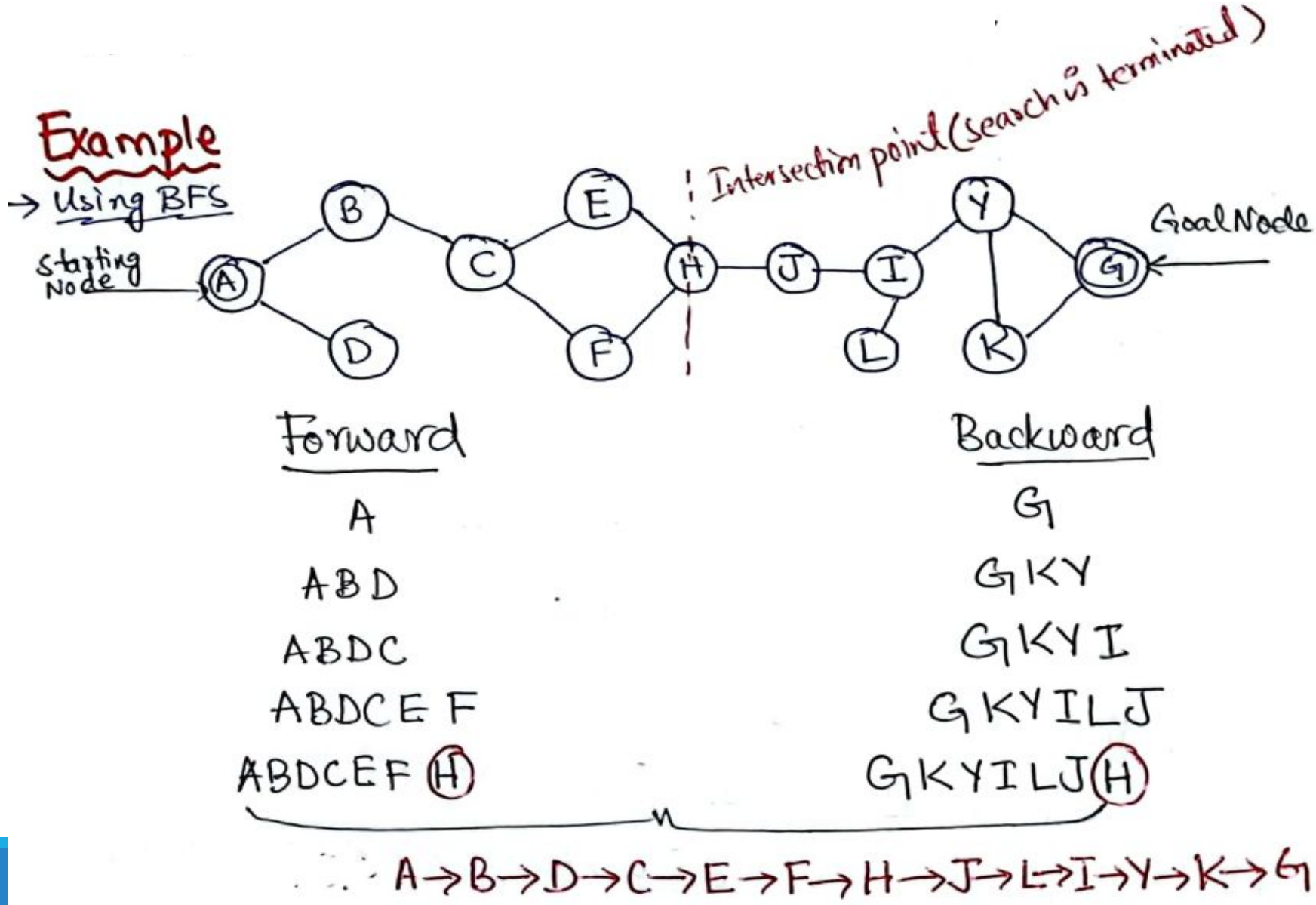
ii) Requires less memory.

Disadvantages:

i) Implementation is difficult. (two different data structures are maintained).

ii) Goal state should be known in advance.

Bi-directional Search Algorithm (4/4)



Iterated Deepening Depth-First Search (1/2)

6) Iterative Deepening Depth-First Search:

↳ Combination of both DFS and BFS.

↳ Best Depth limit is found out by gradually increasing limit. Initial depth limit is '0'. On every iteration increase depth by 1.

↳ Stack is used for DFS part.

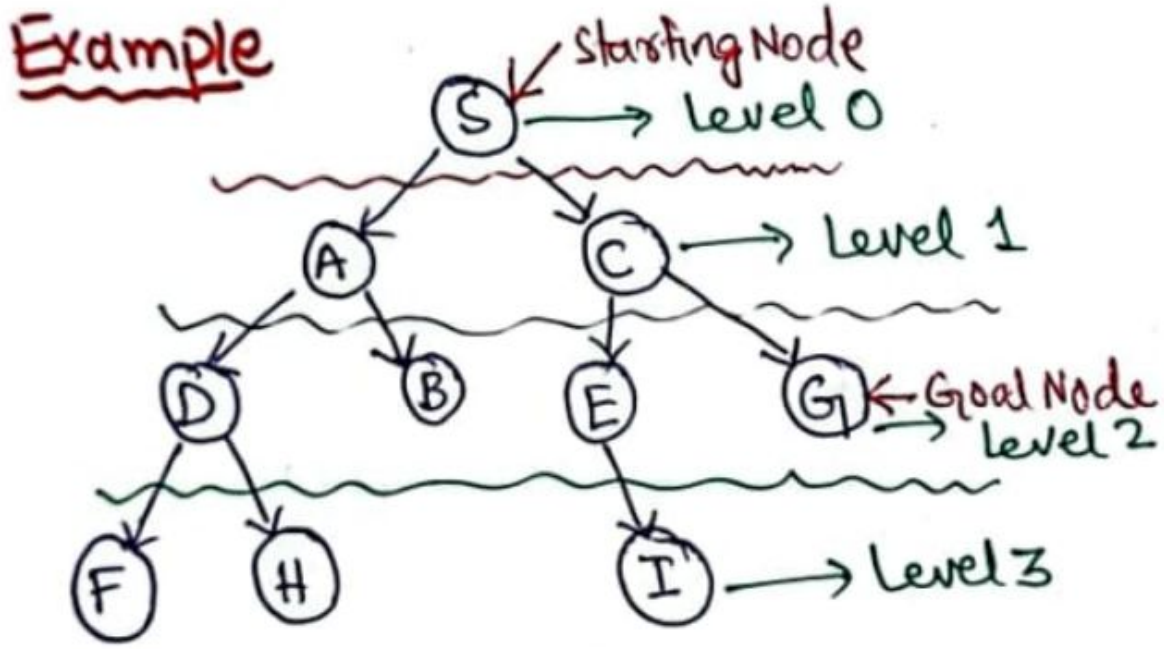
Advantage:

↳ Includes benefits of both BFS & DFS. {Fast Searching or less memory requirements}.

Disadvantage:

↳ Repeats the work of previous phase. in each iteration.

Iterated Deepening Depth-First Search (2/2)



* 1st Iteration, d=0 [S]
* 2nd Iteration, d=0+1=1 [S → A → C]
* 3rd Iteration, d=1+1=2 [S → A → D → B → C → E → G]

References: Textbooks

1. *"Artificial Intelligence", by Elaine Rich and Kevin Knight, (2006), McGraw Hill companies Inc., Chapter 1-22 page 1-613.*
2. *"Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig, (2010), Prentice Hall, Chapter 1-27. page 1-1151.*
3. *"Computational Intelligence: A logical Approach", by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 1-12, page 1-608.*
4. *"Artificial Intelligence: Structures and Strategies for Complex Problem Solving", by George F.Lugar, (2002), Addison-Wesley, Chapter 1-16, page 1-743.*
5. *"AI: A new Synthesis", by Nils J.Nilsson, (1998), Morgan Kaufmann Inc., Chapter 1-25, Page 1-493.*
6. *"Artificial Intelligence: Theory and Practice" by Thomas Dean, (1994), Addison-Wesley, Chapter 1-10, Page 1-650.*
7. *Related documents from open source, mainly internet.*